Na czym polega wytwarzanie sterowane testami? Podstawowe praktyki z nim związane.

[Test-driven development] To technika tworzenia oprogramowania wpisująca się w Agile, która polega na tworzeniu testów przed przystąpieniem do pisania właściwego kodu.

1. Przygotowanie testu jednostkowego.
2. Napisanie podstawowego kodu – tylko tyle, żeby pomyślnie zaliczyć test.
3. Zapewnienie, że kod przechodzi wszystkie poprzednie testy.
4. Dostosowanie kodu do wymagań, zapewnienie poprawnej struktury (refactoring).

Potencjalnie poprawia to jakość kodu i zmniejsza liczbę niewykrytych błędów. Konieczne do tego jest jednak konsekwentne podporządkowanie tej strategii przez cały zespół i nie wprowadzanie zbyt dużych pakietów testów na raz. A przede wszystkim testy powinny być dobrze napisane.

Dobre praktyki TDD:

- Niezależne testy - wynik wykonania testów powinien być niezależny od ich kolejności,
- Czytelne testy - powinny być czytelne, żeby służyć za dokumentację kodu,
- Testy konkretnych funkcjonalności - nie powinny być zbyt złożone, jedna funkcjonalność per test,
- Częste wykonywanie testów w ramach ciągłej integracji oprogramowania.

Na czym polega wstrzykiwanie zależności? Dlaczego ułatwia testowanie i integrację składników systemu?

Wstrzykiwanie zależności to wzorzec projektowy polegający na usuwaniu bezpośrednich zależności pomiędzy obiektami. Dostarczamy potrzebne obiekty (zależności) zamiast konstruowania ich w obiekcie, który ich potrzebuje.

Realizacje wstrzykiwania zależności:

- poprzez konstruktor - obiekt dostaje konstruktor klasy
- poprzez setter - klient udostępnia setter, który służy do wstrzyknięcia zależności
- poprzez interfejs - zależność udostępnia metodę, która wstrzykuje zależność do przekazanego obiektu

Wstrzykiwanie zależności ułatwia modyfikację klas bez konieczności dostosowywania tych, które od nich zależą. Umożliwia też testowanie poszczególnych komponentów w odizolowaniu od innych, ponieważ można łatwiej „mockować” zależności. Bez tego, testując klasę, konieczne byłoby testowanie wszystkich obiektów, które tworzy.

Co to jest wzorzec projektowy? Dwa przykłady wzorców służących do eliminacji/odwracania zależności.

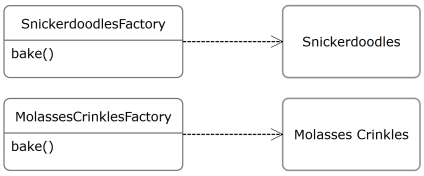
Wzorzec projektowy to rekomendowane rozwiązanie pewnego problemu powszechnie występującego w projektowaniu oprogramowania. Wzorzec jest ogólnym, sposobem implementacji, który można dostosować do konkretnego przypadku.

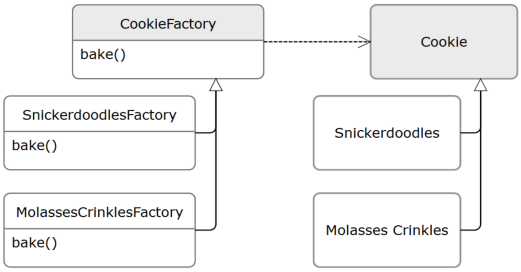
Wzorce projektowe:

- ⊕ porządkują zależności,
- ⊕ wyodrębniają komponenty o różnych funkcjonalnościach, które można łączyć i wykorzystywać ponownie,
- ⊖ zwiększają nakład pracy,
- ⊖ powstrzymują przed szukaniem bardziej nowatorskich rozwiązań.

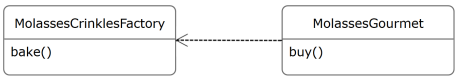
Przykłady wzorców do kontroli zależności

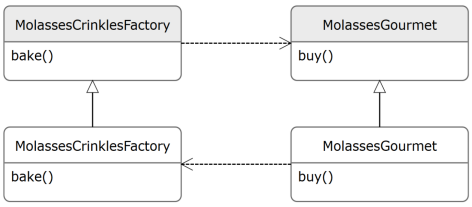
Fabryka Abstrakcyjna:
Służy on do tworzenia rodzin podobnych obiektów, unikając specyfikacji konkretnego typu tworzonego obiektu. Usuwa zależności komponentu wysokiego poziomu od komponentu niższego poziomu poprzez przeniesienie zależności na interfejsy.





Obserwator:
Definiuje zależność jeden-do-wielu pomiędzy obiektami – gdy jeden obiekt zmienia stan, wszystkie obiekty zależne są automatyczne powiadamiane. Eliminuje bezpośrednią zależność pomiędzy obiektem obserwowanym a jego obserwatorami, jest ona przeniesiona na poziom abstrakcji.





Zasady SOLID. Omówienie wybranych dwóch na przykładach.

Zostały wprowadzone przez Roberta C Martina, żeby zachęcać do tworzenia bardziej zrozumiałego i elastycznego oprogramowania, szczególnie przy rozrastaniu się projektu. Oto 5 zasad:

- **Zasada pojedynczej odpowiedzialności (single responsibility)** - klasa powinna mieć tylko jeden powód do zmian, czyli realizować jedną funkcjonalność
- **Zasada otwarte/zamknięte (open/closed)** - klasy powinny być otwarte na rozszerzanie a zamknięte na modyfikacje, czyli dać się łatwo rozszerzać bez konieczności wprowadzania zmian w oryginalnej klasie
- **Zasada podstawienia Liskov (liskov substitution)** - każda klasa pochodna powinna móc zastąpić swoją klasę bazową
- **Zasada segregacji interfejsów (interface segregation)** - klient nie powinien musieć implementować interfejsu, ani być zależnym od metod, których nie używa.
- **Zasada odwrócenia zależności (dependency inversion)** - moduły wysokiego poziomu nie powinny zależeć od tych niższego poziomu a wszystkie powinny zależeć od abstrakcji. Konkretne implementacje powinny zależeć od abstrakcji, nie w przeciwnym kierunku.

Przykłady naruszeń

Zasada pojedynczej odpowiedzialności:
Klasa wykonuje pewne obliczenia, a następnie zwraca wynik w jednym z wybranych formatów (np. JSON lub HTML). Oprócz funkcji odpowiadającej za poprawne wyznaczenie wyniku zawiera logikę związaną z zapisaniem go w odpowiednim formacie, co narusza zasadę SRP. Należy więc stworzyć osobną klasę, która przyjmuje wynik i zwraca go w wybranym formacie.

Zasada segregacji interfejsów:
Chcąc zaimplementować interfejs `Animal`, który dostarcza metod: `walk`, swim, `fly` trzeba ograniczyć się tylko do tych zwierząt, które mają te wszystkie umiejętności, czyli do kaczek oraz kaczkopodobnych. To łamie zasadę ISP. Lepszym rozwiązaniem jest więc podzielenie funkcjonalności na trzy interfejsy `WalkingAnimal`, `SwimmingAnimal`, `FlyingAnimal` i rozszerzanie dowolnej kombinacji z nich wedle potrzeb.